

组合组件ComposeView

🕒 大约 4 分钟 📄 约 1244 字

在 Kuikly 编写UI时, 一个比较好的实践是将页面UI分模块, 然后每个UI模块封装成独立的UI组件, 以达到**UI代码复用**和**UI逻辑分治**的目的。

举个例子: 每个页面都会有**title bar**, 此时我们就可以将**title bar**封装成 `ComposeView` , 然后暴露给各个 Kuikly 页面使用

如何封装ComposeView

我们以**title bar**作为例子, 讲述如何封装**ComposeView**

1. 新建 NavigationBarView 并继承 ComposeView

反馈文档建议

```
internal class NavigationBarView : ComposeView<ComposeAttr, ComposeEvent>() {  
  
    override fun body(): ViewBuilder {  
    }  
  
    override fun createAttr(): ComposeAttr {  
        return ComposeAttr()  
    }  
  
    override fun createEvent(): ComposeEvent {  
        return ComposeEvent()  
    }  
  
}
```

kt

在 `NavigationBarView` 中, 我们需要实现3个方法:

- **body()**: 与 `Pager` 的 `body` 方法作用一样, 在这里我们需要返回描述组合组件的UI结构闭包
- **createAttr()**: 返回组合组件支持的属性类。这里我们在 `NavigationBarView` 指定了组合组件的attr类型为 `ComposeAttr`

- **createEvent()**: 返回组合组件支持的事件类。这里我们在 `NavigationBarView` 指定了组合组件的event类型为 `ComposeEvent`

2. 接着我们实现 `NavigationBarView` 的UI, `NavigationBarView` 左边有一个返回箭头, 中间有一个title

```
internal class NavigationBarView : ComposeView<ComposeAttr, ComposeEvent>() {  
    override fun createAttr(): ComposeAttr {  
        return ComposeAttr()  
    }  
  
    override fun createEvent(): ComposeEvent {  
        return ComposeEvent()  
    }  
  
    override fun body(): ViewBuilder {  
        return {  
  
            View {  
                attr {  
                    size(pagerData.pageViewWidth, 44f)  
                    marginTop(pagerData.statusBarHeight)  
                    allCenter()  
                    backgroundColor(Color.GRAY)  
                }  
  
                Image {  
                    attr {  
                        size(16f, 16f)  
                        src(BASE_64)  
                        resizeContain()  
                        absolutePosition(left = 15f, top = (44f - 16f) / 2)  
                    }  
                }  
  
                Text {  
                    attr {  
                        fontWeightBold()  
                        fontSize(16f)  
                        text("这是标题栏")  
                    }  
                }  
            }  
        }  
    }  
}
```

kt

```

        }

    }

}

companion object {
    private const val BASE_64 =
""
}

}

```

3. 实现完UI后，我们需要提供一个声明式方法给外部业务使用

```

internal fun ViewContainer<*, *>.NavBar(init: NavigationBarView.() -> Unit) {
    addChild(NavigationBarView(), init)
}

```

上述代码中，我们在 ViewContainer 扩展了 NavBar 方法，并传入 NavigationBarView 的初始化闭包，在方法内，调用 addChild ，把 NavigationBarView 实例和初始化闭包传入

4. 外部调用 NavBar 方法即可将title bar组件添加到UI结构上

```

@Page("1")
internal class TestPage : BasePager() {

    override fun body(): ViewBuilder {
        val ctx = this
        return {
            NavBar { }
        }
    }

}

```

组合组件如何接收外部参数

在上面的例子中, title bar的标题我们是写死为"这是标题栏", 外部使用方还无法自定义标题栏的标题。那在组合组件中, 如何接收外部传入的参数呢?

每个组合组件都有一个 Attr 类, 代表组件自身的属性, 让外部在调用组合组件时, 配置组合组件的参数, 还是以我们title bar作为例子, 来看如何将标题名称改造成支持外部传入

1. 首先新建 NavBarAttr , 然后继承 ComposeAttr , 并在其中定义 title 属性

```
internal class NavBarAttr : ComposeAttr() {
```

kt

```
    var title = ""
```

反馈
文档
建议

2. 接着我们来 NavigationBarView 中, 将 Attr 的类型指定为 NavBarAttr , 并在 createAttr 方法中返回 NavBarAttr 实例, 最后将 NavBarAttr 中的title设置给Text组件

```
internal class NavigationBarView : ComposeView<NavBarAttr, ComposeEvent>() {
```

kt

```
    override fun createAttr(): NavBarAttr {
```

```
        return NavBarAttr()
```

```
    }
```

```
    override fun createEvent(): ComposeEvent {
```

```
        return ComposeEvent()
```

```
    }
```

```
    override fun body(): ViewBuilder {
```

```
        val ctx = this
```

```
        return {
```

```
            View {
```

```
                attr {
```

```

        size(pagerData.pageViewWidth, 44f)
        marginTop(pagerData.statusBarHeight)
        allCenter()
        backgroundColor(Color.GRAY)
    }

    Image {
        attr {
            size(16f, 16f)
            src(BASE_64)
            resizeMode()
            absolutePosition(left = 15f, top = (44f - 16f) / 2)
        }
    }

    Text {
        attr {
            fontWeightBold()
            fontSize(16f)
            text(ctx.attr.title)
        }
    }
}

}

}

}

companion object {
    private const val BASE_64 =
""
}

}

```

3. 外部在使用 NavBar 时, 可在 attr{} 中, 配置标题

```

@Page("1")
internal class TestPage : BasePager() {

    private var translateAnimationFlag by observable(false)

    override fun body(): ViewBuilder {

```

kt

```

        val ctx = this
        return {
            NavBar {
                attr {
                    title = "外部传入的标题"
                }
            }
        }
    }
}

```

外部如何监听组合组件的事件

在上面的例子中，我们 NavBar 左边有一个返回箭头，每个使用方对返回箭头的点击处理可能不一样，那外部在使用组合组件 NavBar 时，如何监听返回箭头的点击事件呢？

 组合组件都有一个 Event 类，代表组合组件自身支持的事件，外部方在使用组合组件可通过 event{} 来监听组合组件的事件。下面我们还是以title bar作为例子，来看如何外部使用方监听返回箭头的点击事件

1. 首先新建 NavBarEvent ，然后继承 ComposeEvent ，并在其中定义 backIconClick 方法

```

internal class NavBarEvent : ComposeEvent() {

    var clickHandler: (() -> Unit)? = null

    fun backIconClick(handler: () -> Unit) {
        clickHandler = handler
    }
}

```

kt

2. 接着我们来 NavigationBarView 中，将 Event 的类型指定为 NavBarEvent ，并在 createEvent 方法中返回 NavBarEvent 实例，最后在返回箭头icon被点击时，通知外部

```
internal class NavigationBarView : ComposeView<NavBarAttr, NavBarEvent>() {

    override fun createAttr(): NavBarAttr {
        return NavBarAttr()
    }

    override fun createEvent(): NavBarEvent {
        return NavBarEvent()
    }

    override fun body(): ViewBuilder {
        val ctx = this
        return {

            View {
                attr {
                    size(pagerData.pageViewWidth, 44f)
                    marginTop(pagerData.statusBarHeight)
                    allCenter()
                    backgroundColor(Color.GRAY)
                }

                Image {
                    attr {
                        size(16f, 16f)
                        src(BASE_64)
                        resizeMode()
                        absolutePosition(left = 15f, top = (44f - 16f) / 2)
                    }

                    event {
                        click {
                            ctx.event.clickHandler?.invoke() // 回调给外部
                        }
                    }
                }

                Text {
                    attr {
                        fontWeightBold()
                        fontSize(16f)
                        text(ctx.attr.title)
                    }
                }
            }
        }
    }
}
```

```

        }

    }

}

companion object {
    private const val BASE_64 =
""
}

}

```

3. 外部在使用 NavBar 时,可在 evnet{} 中, 监听 NavBar 组件的返回箭头点击事件

```
internal class TestPage : BasePager() {
```

kt

```

    override fun body(): ViewBuilder {
        val ctx = this
        return {
            NavBar {
                attr {
                    title = "外部传入的标题"
                }

                event {
                    backIconClick {
                        // 返回键点击事件
                    }
                }
            }
        }
    }
}

```


下一步

在了解并学习了 Kuikly 中的**组合组件概念以及用法**后, 接下来我们来学习组合组件 [ComposeView生命周期](#)